Búsquedas heurísticas y optimización

Introducción

Imagina que debes planificar la ruta óptima para un vehículo de reparto que atiende treinta domicilios en una ciudad. Si se intentara evaluar todas las combinaciones posibles de recorrido, se enfrentaría lo que matemáticamente se conoce como el **problema del viajante de comercio**. Con apenas treinta destinos, el número de rutas posibles supera los 10^{30}. Para contextualizarlo, incluso evaluando un millón de rutas por segundo, se necesitaría más tiempo del que lleva el universo en existir.

> Ese es, en esencia, el problema que motivó el desarrollo de las heurísticas: encontrar soluciones suficientemente buenas en tiempo razonable, cuando las soluciones perfectas resultan computacionalmente inaccesibles.

Y este desafío no pertenece exclusivamente al mundo de la logística. Quienes se dediquen a la ciencia de datos lo encontrarán al ajustar hiperparámetros; los especialistas en seguridad informática lo reconocerán en algoritmos de detección de amenazas; los bioinformáticos lo verán en el alineamiento de secuencias genéticas; los ingenieros de robótica lo resolverán al planificar trayectorias en entornos dinámicos.

La heurística, entonces, es una competencia profesional transversal. Russell y Norvig (2016) dedican capítulos de su libro a los algoritmos de búsqueda informada, argumentando que el conocimiento aproximado del dominio es precisamente lo que distingue a un sistema inteligente de uno que simplemente explora por fuerza bruta. Esta lectura está organizada para construir comprensión de manera progresiva.

Se comenzará con el concepto de heurística y sus propiedades formales. Luego se examinarán los algoritmos de búsqueda voraz y A* con nivel de detalle suficiente para trazar su ejecución sobre

ejemplos concretos. Luego se trabajará con el dilema exploración-explotación como principio unificador y, finalmente, se revisarán aplicaciones concretas.

Caso: búsqueda heurística en la protección de infraestructura

Para entender el impacto real de las búsquedas heurísticas, conviene detenerse en un caso de ciberseguridad industrial. Imagina un software diseñado para proteger la red eléctrica de una ciudad con un sistema que supervisa más de diez mil dispositivos interconectados, desde computadoras de oficina hasta los interruptores físicos que controlan la alta tensión en las subestaciones.

Ante cualquier alerta técnica o comportamiento anómalo, el sistema debe rastrear el origen del ataque y devolver una lista de los puntos en mayor peligro, ordenados por prioridad. Tiene que hacerlo en milisegundos, porque en un ciberataque el daño se propaga muy rápidamente.

> Este escenario replica con precisión la estructura de un problema de búsqueda en espacio de estados. El espacio de estados son los diez mil dispositivos que conforman la red. El estado inicial es el conjunto de alarmas y datos inusuales que los sensores acaban de registrar. El estado objetivo es identificar el foco principal del ataque, algo que al comienzo de la crisis se desconoce por completo. Las acciones, por último, son las conexiones lógicas que permiten a un atacante saltar de un dispositivo a otro.

Lanzar una búsqueda exhaustiva sobre este mapa sería computacionalmente inviable. Evaluar todos los órdenes posibles en que inspeccionar diez mil dispositivos equivale a calcular 10.000!, un número que ronda 10^35.659. Para dimensionarlo, el total de átomos del universo observable apenas alcanza 10^80. Ningún sistema podría recorrer ese espacio en tiempo útil.

Pero la inviabilidad no es solo computacional, sino también es operativa. Analizar cada componente con la misma importancia generaría un listado caótico donde un intento fallido de contraseña en una oficina administrativa tendría la misma relevancia que un comando de apagado dirigido a una turbina central. En una emergencia, si todo es prioritario, nada lo es. El valor del sistema depende de que lo verdaderamente grave aparezca al principio de la pantalla.

Para resolver esto, el software implementa una función heurística que evalúa el nivel de amenaza de cada dispositivo mediante un puntaje combinado. Por un lado, mide el costo real: la cantidad de alertas técnicas concretas que el dispositivo ya está disparando. Por el otro, suma el valor heurístico propiamente dicho: una estimación del daño potencial basada en el perfil de la red. Esta estimación considera qué tan importante es ese componente para el funcionamiento de la ciudad, si tiene salida directa a internet y con qué rapidez podría contagiarse el resto del sistema si ese nodo cae. El algoritmo ordena la búsqueda expandiendo primero los caminos con los puntajes más altos.

Lo más interesante del sistema es la decisión que toma respecto a la optimalidad. En teoría, un buen algoritmo debería encontrar siempre la solución más precisa; identificar exactamente los dispositivos comprometidos, sin incluir falsos positivos. Sin embargo, este software está diseñado para hacer lo contrario, ya que ante cualquier ambigüedad, asume el peor escenario posible y magnifica el nivel de riesgo de los componentes más críticos.

Es una renuncia a la optimalidad y tiene un costo y un beneficio concretos. El costo es que la lista de alertas será más larga de lo necesario e incluirá dispositivos que quizás solo sufrían un parpadeo en la red. El beneficio es que ningún peligro real quedará oculto al fondo de la pantalla. Un operador humano en el turno noche podría ignorar una anomalía leve en una subestación central: el sistema se encarga de que eso no ocurra, colocándola arriba de la lista aunque la amenaza resulte menor de lo estimado. En este dominio, una lista más larga de lo ideal es un inconveniente menor. Pasar por alto un ataque a la red eléctrica de una ciudad, no.

A modo de reflexión

El sistema que se analizó prioriza no omitir ninguna amenaza, aunque eso signifique generar alertas de más. Pero imagina el escenario inverso: una red hospitalaria donde cada alerta falsa interrumpe un quirófano en operación o desconecta equipos de soporte vital.

> ¿Debería ese sistema usar la misma lógica? ¿Qué cambiaría en el diseño de la función heurística si el costo de una falsa alarma fuera tan grave como el costo de una omisión?

Anota tu respuesta. Se retomará esta pregunta al final de la lectura para ver cómo evolucionó tu perspectiva.

Concepto de heurística

La palabra 'heurística' proviene del griego heuriskein, que significa encontrar, descubrir. En el contexto de la inteligencia artificial, designa una función o regla que proporciona una estimación del costo o valor de un estado, sin garantizar que dicha estimación sea exacta.

> *Russell y Norvig (2016) definen formalmente la función heurística h(n) como una estimación del costo desde el nodo n hasta el objetivo más próximo, subrayando que su utilidad radica precisamente en su eficiencia computacional, no en su exactitud.*

¿Para qué se necesita heurísticas?

Para entender por qué las heurísticas importan, es necesario recuperar el concepto de explosión combinatoria que se estudió en el módulo anterior. Los algoritmos de búsqueda no informada exploran el espacio de estados de forma ciega, sin ningún conocimiento sobre cuáles estados son más prometedores que otros.

Se considera un grafo con un factor de ramificación b (cada nodo tiene en promedio b sucesores) y una profundidad de solución d. La búsqueda en anchura deberá explorar en el peor caso todos los nodos hasta la profundidad d, lo que equivale a O(b^d) nodos. Con b = 10 y d = 10, eso son diez mil millones de nodos. Con d = 20, son 10^{20} nodos, número que supera la capacidad de cualquier computadora concebible. Eso es la explosión combinatoria, y es la razón principal por la que se necesita guiar la búsqueda con conocimiento del dominio.

Tabla 1. Explosión combinatoria en búsqueda no informada

Factor b	Profundidad d	Nodos BFS (aprox.)	Tiempo estimado (1M nodos/seg)	¿Tratable?
10	5	100.000	0,1 segundos.	Sí.
10	10	10.000.000.000	~3 horas.	Difícil.
10	15	1.000.000.000.000.000	> 31 años.	No.
10	20	10^{20} nodos	Millones de años.	Imposible.

Fuente: elaboración propia con base en Rusell y Norvig (2016).

La heurística actúa como un mapa conceptual que orienta la búsqueda hacia las regiones del espacio que, con mayor probabilidad, contienen buenas soluciones. En lugar de explorar sistemáticamente todos los nodos, el algoritmo prioriza aquellos que la heurística estima como más prometedores. El resultado, en la práctica, puede ser una reducción fuerte del número de nodos explorados: de millones a miles, o incluso a cientos.

En síntesis, lo anterior significa que, usando buenas heurísticas, aunque ocasionalmente una buena ruta sea pasada por alto, se obtendrán soluciones apropiadas (no necesariamente óptimas) para problemas difíciles, con demandas de recursos compatibles con las posibilidades. Y eso, en la mayoría de los contextos prácticos, es exactamente lo que se necesita.

Propiedades de una buena función heurística

Masip, Llorà y Domingo (2015) identifican tres propiedades que determinan la calidad de una función heurística. Comprenderlas en profundidad es fundamental para diseñar o evaluar heurísticas en aplicaciones reales.

- En primer lugar, **la admisibilidad establece un límite para la estimación**. Se dice que una función heurística es admisible si nunca sobreestima el costo real para alcanzar el objetivo desde el nodo actual. En otras palabras, la estimación del algoritmo siempre es menor o igual al costo real exacto del camino óptimo.

- **Avanzando en el nivel de exigencia, se encuentra la consistencia**. Una heurística se considera consistente si la estimación del nodo actual no supera el costo de moverse al siguiente nodo más la estimación de ese nuevo nodo. Existe una regla de implicación directa entre ambas propiedades: toda heurística consistente es necesariamente admisible, pero no toda heurística admisible es consistente.

- Finalmente, una vez asegurada la validez mediante las propiedades anteriores, se evalúa la **informatividad (o relación de dominancia)** para optimizar el rendimiento. Dadas dos heurísticas que ya son admisibles, se dice que la segunda domina a la primera si sus estimaciones son siempre mayores o iguales para todos los nodos. En términos prácticos, una heurística más informativa ofrece valores más altos y cercanos al costo real sin llegar a pasarse. Esto se traduce en una búsqueda mucho más eficiente, ya que el algoritmo descarta más caminos incorrectos y expande menos nodos en total.

Figura 1: Propiedades que indican la calidad de la función heurística



Fuente: elaboración propia.

Para plantear un caso concreto, se comparan tres heurísticas para el clásico puzzle-8, que consiste en un tablero 3×3 con 8 piezas numeradas y un espacio vacío que hay que ordenar moviendo piezas al espacio libre.

La idea central detrás de las tres heurísticas es que todas miran el mismo tablero desordenado, pero cada una captura una cantidad distinta de información sobre cuánto trabajo falta.

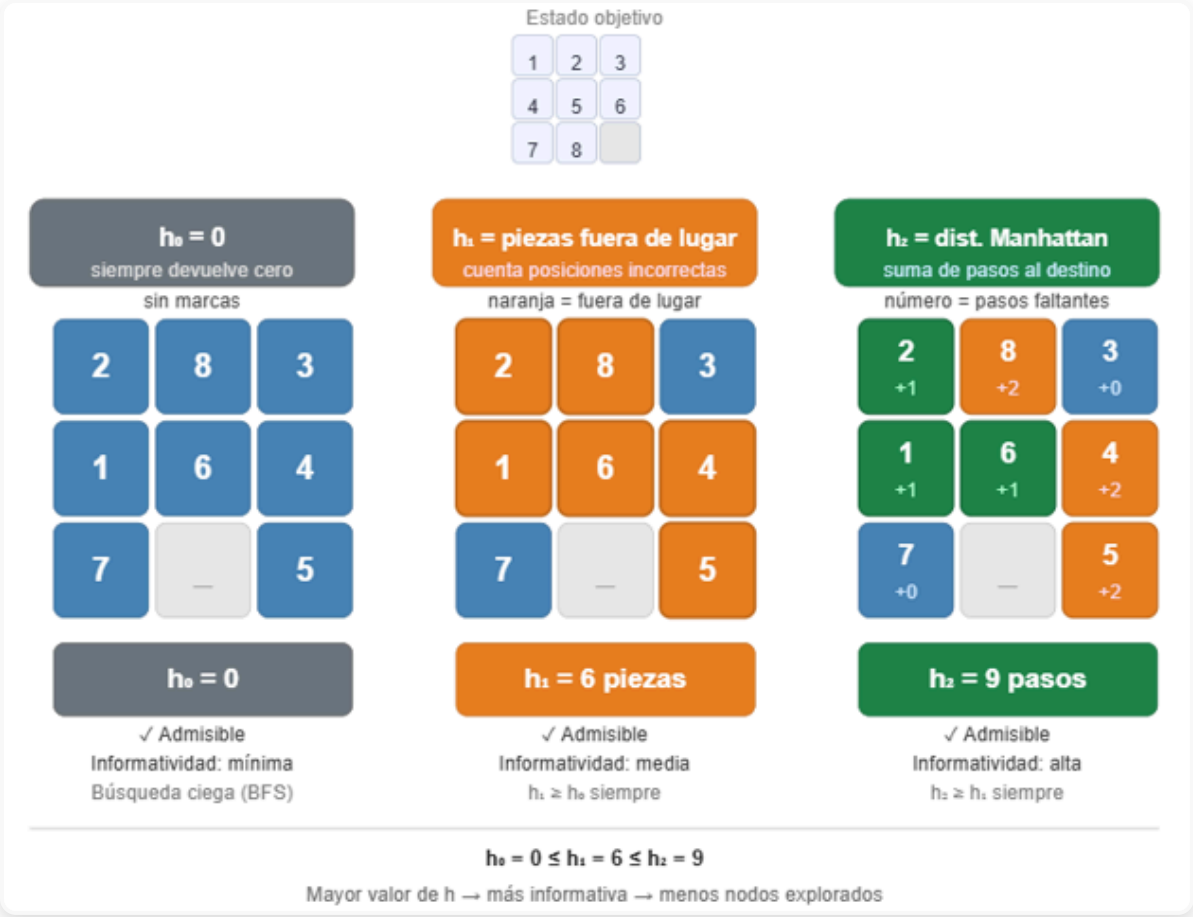
- **h_0 es el caso en el cual no sabe nada del estado actual**. El algoritmo trata todos los caminos como igualmente prometedores y termina explorando el espacio de búsqueda a ciegas, como si fuera BFS. Es útil como punto de referencia teórico, no como herramienta práctica.

- **h_1 da un primer paso hacia la inteligencia**. Detecta que algo está mal, aunque no puede decir cuánto. Sabe que la pieza 8 está fuera de lugar, pero ignora si le falta un movimiento o cinco para llegar a destino. Por eso subestima sistemáticamente el costo real, lo cual la mantiene admisible, pero la hace poco precisa.

> **h_2 corrige exactamente esa ceguera**. En lugar de preguntar ¿está bien o mal?, pregunta ¿a qué distancia está? La diferencia es sutil, pero tiene consecuencias enormes en la práctica. Al estimar costos más ajustados a la realidad, el algoritmo A* (que ya se va a analizar en detalle) puede descartar ramas enteras del árbol de búsqueda que h_1 habría seguido explorando.

La relación $h_2 \geq h_1 \geq h_0$ no es una coincidencia del ejemplo, sino una propiedad matemática garantizada. Una heurística que domine a otra siempre produce un A* igual o más eficiente, porque nunca expande un nodo que la heurística dominante ya descartó. Esto hace que la elección de heurística sea directamente una decisión sobre cuánta memoria y tiempo se va a consumir.

Figura 2. Ejemplo de tres heurísticas para el puzzle-8



Fuente: elaboración propia.

Diseño de heurísticas

Una estrategia para diseñar heurísticas admisibles es la relajación del problema, es decir, crear una versión simplificada del problema original eliminando alguna de sus restricciones. El costo óptimo en el problema relajado siempre será menor o igual al costo en el problema real, porque se permiten más acciones, por lo que la solución óptima del problema relajado es automáticamente una heurística admisible del original.

Volviendo al puzzle-8: si se relaja la restricción de que las piezas solo pueden moverse al espacio vacío y permitir que cualquier pieza se mueva a cualquier posición adyacente aunque esté ocupada, se obtendrá exactamente la distancia de Manhattan. Si se relaja aún más y se permite que las piezas se muevan a cualquier posición en el tablero directamente, se obtendrá h_1 (número de piezas fuera de lugar). Ambas son heurísticas admisibles, derivadas sistemáticamente por relajación.

Experimentación: diseño de heurística

- ✓ Plataforma: Registrar cuál IAG se utiliza.
- ✓ Prompt sugerido: soy estudiante de [tu carrera]. Necesito diseñar una función heurística admisible para el siguiente problema de mi campo: [describe un problema de búsqueda u optimización de tu disciplina]. Explícame: (1) qué es el costo real h*(n) que estamos estimando, (2) cómo aplicaría el método de relajación del problema para derivar una heurística admisible, (3) por qué esa heurística nunca sobreestima el costo real, y (4) cómo podría hacer la heurística más informativa sin violar la admisibilidad.
- ✓ Reflexión: ¿La heurística propuesta es admisible? ¿Qué pasa si la heurística se sobreestima en algunos casos? ¿La búsqueda seguiría siendo completa? ¿Y

óptima?

Algoritmos de búsqueda heurística

Con el concepto de heurística claro, es posible analizar cómo se integra en algoritmos de búsqueda concretos. La diferencia central respecto a los métodos de búsqueda a ciegas es que, ahora, a cada nodo se le asocia un valor que da idea de lo cerca que se encuentra de un nodo meta.

Ese valor no es más que una estimación de la distancia real, y la incorporación de este tipo de información, normalmente a través de las llamadas funciones de evaluación heurística, permite guiar la búsqueda por caminos evaluados como los más prometedores.

La función heurística genérica f(n) puede descomponerse en dos partes: la primera depende de la distancia del nodo n al estado inicial (g), y la segunda, de su distancia al estado objetivo (h). Es decir:

$$f(n) = g(n) + h(n)$$

Donde g(n) es el costo estimado del camino ya recorrido hasta el nodo n, y h(n) es la distancia estimada que separa el nodo n del estado objetivo.

Nótese que g(n) = 0 cuando n es el nodo inicial, y h(n) = 0 cuando n es el nodo objetivo. También es importante reconocer la existencia de un valor real f*(n) = g*(n) + h*(n) que generalmente no es conocido. La función f(n) representa la mejor aproximación posible, con la condición g(n) ≤ g*(n) y h(n) ≤ h*(n).

Búsqueda voraz (Greedy Best-First)

La búsqueda voraz es la versión más directa del uso de heurísticas. En cada paso, selecciona el nodo que la función heurística h(n) estima como más prometedor, sin considerar el costo acumulado para llegar hasta ese punto. La función de evaluación es simplemente la estimación heurística del costo resultante:

$$f(n) = h(n)$$

Tabla 2. Algoritmo: búsqueda voraz (Best-First Greedy)

```
función BÚSQUEDA_VORAZ(problema, h):

frontera ← cola de prioridad ordenada por h(n), con nodo_inicial
visitados ← conjunto vacío

mientras frontera no esté vacía:
nodo ← frontera.extraer_mínimo() // el de menor h(n)

si nodo.estado == problema.objetivo:
devolver RECONSTRUIR_CAMINO(nodo)

si nodo.estado no está en visitados:
visitados.agregar(nodo.estado)

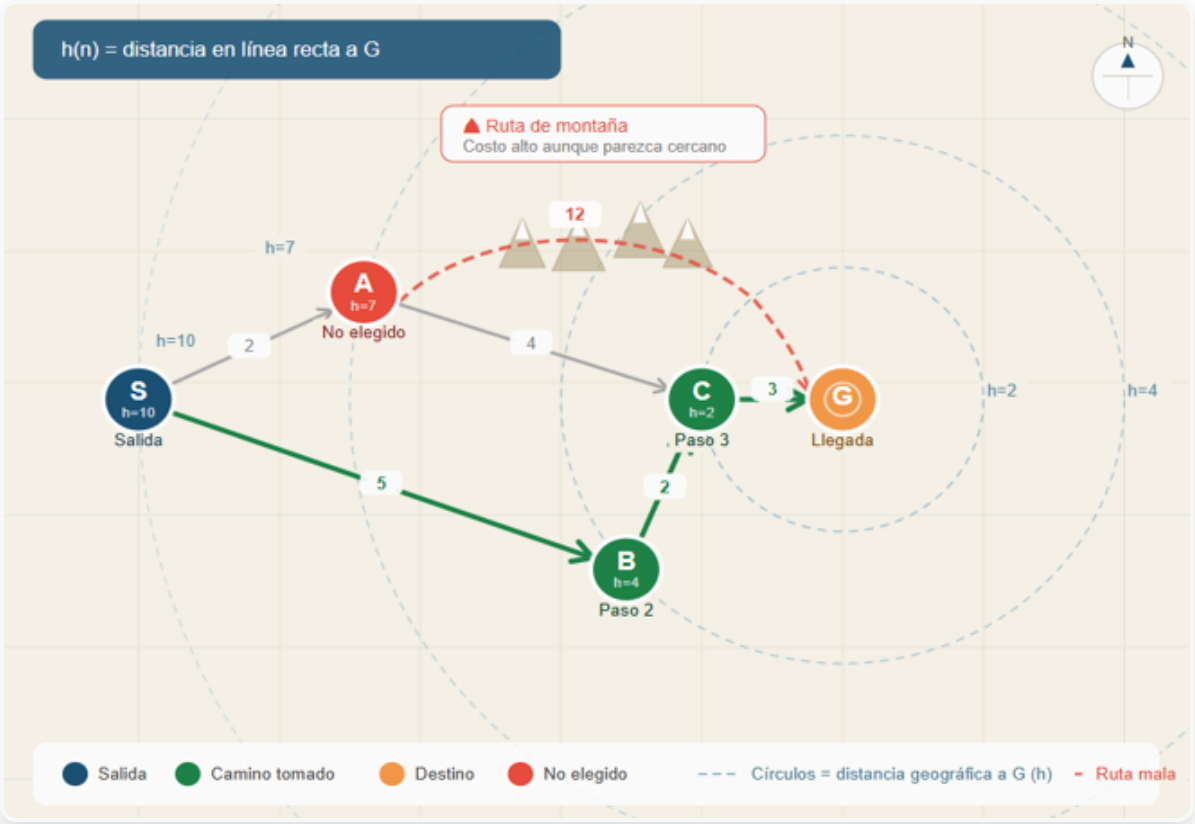
para cada sucesor en EXPANDIR(nodo, problema):
si sucesor.estado no está en visitados:
frontera.insertar(sucesor, prioridad=h(sucesor))

devolver FRACASO // no existe solución
```

Fuente: elaboración propia.

Para entender cómo se comporta, a continuación se presenta un ejemplo concreto. Considera un mapa simplificado de ciudades: S (inicio), A, B, C, G (destino). La heurística h(n) representa la distancia en línea recta estimada al destino G.

Figura 3: Mapa con la representación de Greedy



Fuente: elaboración propia.

Cada círculo representa una distancia geográfica fija desde G. Las ciudades que quedan en el círculo que corresponde a su h(n). B cae en h=4 (más cerca de G en el mapa) y A cae en h=7 (más lejos). ¿Por qué A tiene costo 12 si está a solo h=7 del destino? Porque entre A y G hay una montaña. La distancia en línea recta es corta, pero la ruta real debe rodearla, lo que encarece el viaje.

Tabla 3. Aplicación de Greedy

Paso	Nodo expandido	h(n)	Frontera después (h)
1	S (inicio).	10	A (h=7), B (h=4)
2	B (menor h).	4	A (h=7), C (h=2)
3	C (menor h).	2	A (h=7), G (h=0)

Paso	Nodo expandido	h(n)	Frontera después (h)
4	G (objetivo).	0	Solución encontrada.

Fuente: elaboración propia.

En contextos prácticos, la búsqueda voraz suele usarse cuando se necesita una respuesta rápida y la calidad subóptima es aceptable. Un sistema de detección de intrusiones que usa búsqueda voraz puede identificar la amenaza más probable muy rápidamente, aunque no sea la más peligrosa en términos de impacto total.

A modo de repaso

¿Es óptima esta solución? El camino encontrado fue S→B→C→G con costo 5+2+3 = 10.

Pero el camino S→A→C→G tiene costo 2+4+3 = 9, que es MENOR.

La búsqueda voraz eligió ir primero a B porque tenía menor h, pero pasó por alto que el camino por A, aunque con peor heurística, resultaba más corto en total. Este es el tipo de error típico de la búsqueda voraz: optimiza localmente a expensas de la optimalidad global

Algoritmo A*

El algoritmo A* resuelve la principal debilidad de la búsqueda voraz introduciendo un balance entre el costo acumulado real y la estimación heurística del costo restante. Es uno de los algoritmos más estudiados y usados en IA, introducido por Hart, Nilsson y Raphael en 1968, con aplicaciones que van desde videojuegos hasta planificación de misiones robóticas. Su función de evaluación es:

$$f(n) = g(n) + h(n)$$

Donde g(n) es el costo real acumulado y h(n) es la estimación heurística al objetivo.

Tabla 4. Algoritmo: A* (A-Estrella)

```
función A_ESTRELLA(problema, h):

nodo_inicial ← estado = problema.inicio, g = 0, f = h(inicio)
frontera ← cola de prioridad ordenada por f(n), con nodo_inicial
visitados ← diccionario vacío {estado → costo_g mínimo conocido}

mientras frontera no esté vacía:
nodo ← frontera.extraer_mínimo() // el de menor f(n) = g(n)+h(n)

si nodo.estado == problema.objetivo:
devolver RECONSTRUIR_CAMINO(nodo) // ← solución óptima garantizada

si nodo.estado ya está en visitados con g ≤ nodo.g:
continuar // ya lo procesamos por un camino mejor

visitados[nodo.estado] ← nodo.g

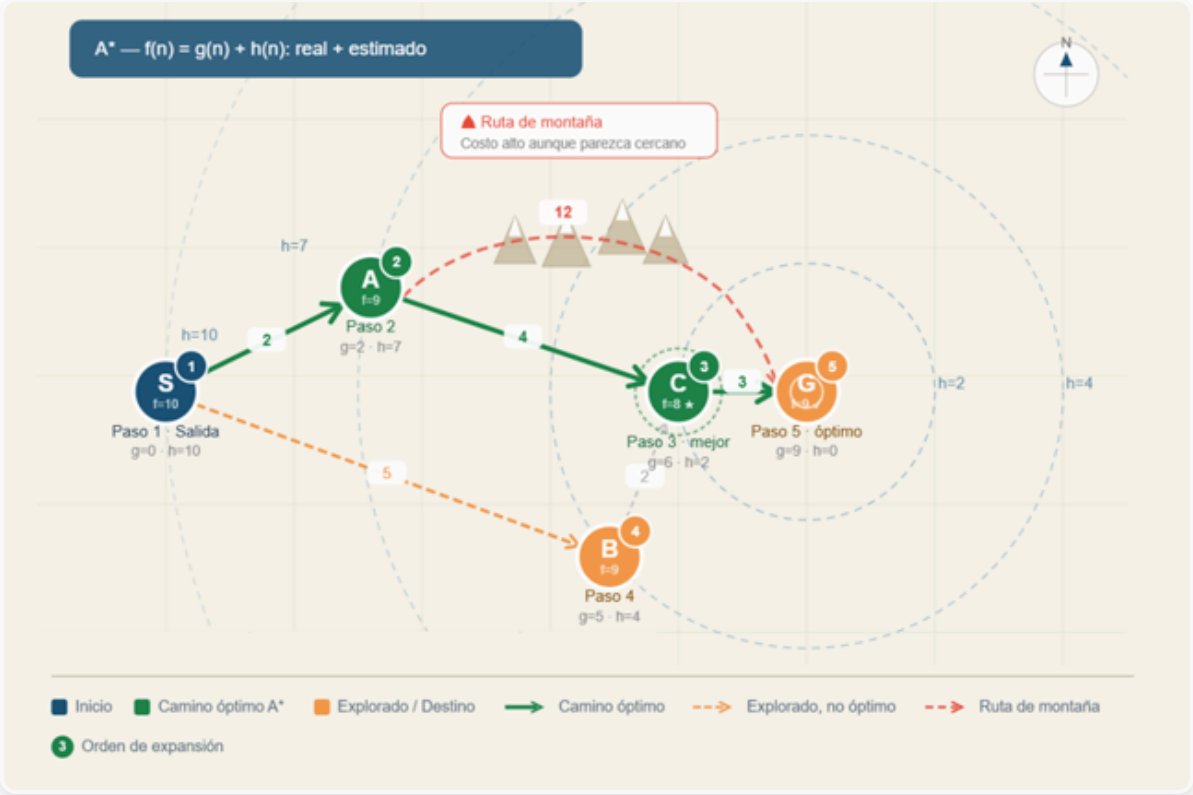
para cada (sucesor, costo_accion) en EXPANDIR(nodo, problema):
g_nuevo ← nodo.g + costo_accion
f_nuevo ← g_nuevo + h(sucesor.estado)
frontera.insertar(sucesor, prioridad=f_nuevo)

devolver FRACASO
```

Fuente: elaboración propia.

Al incluir g_nuevo, A* penaliza los nodos que son aparentemente prometedores (h bajo) pero que están lejos del inicio (g alto). Este balance es lo que garantiza encontrar el camino de menor costo total. Se traza la ejecución sobre el mismo grafo de ciudades para comparar.

Figura 4. Mapa con la representación de A*



Fuente: elaboración propia.

Como diferencia fundamental con Greedy, A* explora el nodo A en el paso 2, aunque tiene peor heurística que B porque su f = 9 es igual al de B. Y al explorar A, descubre el camino a C con f = 8, que es mejor que cualquier otra opción.

La búsqueda voraz nunca consideró ir por A primero porque solo miraba h(n) = 7 frente a h(B) = 4. Esta capacidad de ver el costo total y no solo la distancia estimada restante es lo que distingue a A* y justifica su garantía de optimalidad.

Tabla 5. Aplicación de A*

Paso	Nodo expandido	g(n)	h(n)	f(n)=g+h	Frontera después (nodo: f)
1	S	0	10	10	A: f=2+7=9, B: f=5+4=9
2	A (f=9, menor empate)	2	7	9	B: f=9, C_viaA: f=6+2=8, G_viaA: f=14+0=14

Paso	Nodo expandido	g(n)	h(n)	f(n)=g+h	Frontera después (nodo: f)
3	C (f=8, vía S→A→C)	6	2	8	B: f=9, G_viaA: f=14, G_viaC: f=9+0=9
4	B (f=9)	5	4	9	G_viaA: f=14, G_viaC: f=9, C_viaB: f=7+2=9
5	G (f=9, vía S→A→C→G)	9	0	9	Solución óptima encontrada.

Fuente: elaboración propia.

¿Por qué A* es óptimo?

La garantía de optimalidad de A* puede entenderse intuitivamente. Supón que existe un camino óptimo C* con costo total g*. En el momento en que A* está por expandir el objetivo a través de un camino subóptimo C con costo g > g*, debe haber algún nodo n en la frontera que está en el camino óptimo C*. Para ese nodo n:

$$f(n) = g(n) + h(n) \leq g* + 0 = g* < g$$

Esto significa que f(n) ≤ g*, y como la cola de prioridad ordena por f, el nodo n sería expandido antes que el objetivo subóptimo. Por lo tanto, A* nunca puede expandir el objetivo de forma subóptima si la heurística es admisible (Russell y Norvig, 2016).

Visualiza los diferentes algoritmos de búsqueda

Accede a:

Dibuja un laberinto simple y prueba el algoritmo A*. Luego pruébalo, pero con Greedy Best First y cuenta cuántos nodos explora cada uno.

Reflexión: ¿En qué tipo de laberinto la diferencia entre A* y voraz es mayor? ¿Cuándo son equivalentes? ¿Cuál usarías en un robot de servicio y cuál en un videojuego?

Problemas de optimización

Los algoritmos de búsqueda estudiados hasta ahora tienen un objetivo aparentemente sencillo y es encontrar un camino desde un estado inicial hasta un estado meta. Sin embargo, en la mayoría de los problemas reales no basta con encontrar cualquier solución. Lo que verdaderamente interesa es encontrar una solución que sea mejor que las demás según algún criterio de calidad previamente definido.

Este tipo de situaciones se conoce como problemas de optimización. En ellos existe un conjunto potencialmente muy grande de soluciones válidas y cada una puede evaluarse mediante una función de costo o utilidad. El objetivo ya no consiste únicamente en alcanzar una meta, sino en identificar la alternativa que minimiza o maximiza una determinada medida de desempeño. Dependiendo del dominio, dicha medida puede representar distancia recorrida, tiempo de ejecución, consumo energético, costo económico, utilización de recursos o cualquier otra variable relevante para el problema (Russell y Norvig, 2016).

La noción de optimización aparece de forma natural en una enorme cantidad de aplicaciones. Un sistema de navegación no busca simplemente una ruta entre dos ciudades; intenta encontrar la ruta más conveniente. Una empresa logística no desea únicamente distribuir mercadería; procura hacerlo minimizando costos y tiempos de transporte. Del mismo modo, un planificador industrial no pretende únicamente completar un conjunto de tareas, sino organizarlas de la manera más eficiente posible.

Desde una perspectiva formal, un problema de optimización puede entenderse como la búsqueda de un elemento óptimo dentro de un espacio de soluciones. El desafío radica en que dicho espacio suele crecer de forma exponencial a medida que aumenta la complejidad del problema. Incluso cuando todas las reglas son conocidas y el entorno es completamente determinístico, la cantidad

de alternativas posibles puede resultar tan grande que una exploración exhaustiva se vuelve impracticable.

Esta limitación obliga a los algoritmos a tomar decisiones selectivas durante el proceso de búsqueda. En lugar de examinar todas las posibilidades, deben decidir cuáles parecen más prometedoras y cuáles pueden descartarse temporalmente. En otras palabras, el algoritmo necesita administrar cuidadosamente sus recursos computacionales para maximizar la calidad de la solución obtenida dentro de un tiempo razonable.

Aquí es donde adquieren relevancia las heurísticas. Una heurística proporciona conocimiento adicional sobre la estructura del problema y permite estimar qué estados tienen mayores probabilidades de conducir hacia una buena solución. Gracias a esta información, el algoritmo puede concentrar sus esfuerzos en determinadas regiones del espacio de búsqueda sin necesidad de explorar exhaustivamente todas las alternativas disponibles (Poole y Mackworth, 2017).

Sin embargo, el uso de heurísticas introduce una nueva dificultad. Si el algoritmo confía demasiado en las estimaciones disponibles, corre el riesgo de concentrarse prematuramente en una región del espacio de estados que parece prometedora, pero que en realidad conduce a una solución deficiente. Por el contrario, si intenta analizar demasiadas alternativas para evitar este problema, el costo computacional puede crecer hasta volverse prohibitivo. Esta tensión da origen a uno de los principios fundamentales que atraviesan gran parte de la inteligencia artificial moderna: el equilibrio entre exploración y explotación.

Dilema entre exploración y explotación



Exploración

Haz clic para ver más

(Información del reverso a continuación)

La exploración representa la capacidad del sistema para investigar regiones nuevas, desconocidas o poco analizadas del espacio de estados. Su objetivo es ampliar el conocimiento disponible sobre el problema y descubrir alternativas que podrían conducir a soluciones superiores. Desde esta perspectiva, explorar implica asumir un costo inmediato con la expectativa de obtener beneficios futuros.



Explotación

Haz clic para ver más

(Información del reverso a continuación)

La explotación, en cambio, consiste en aprovechar al máximo la información ya disponible. En lugar de invertir recursos en examinar nuevas posibilidades, el algoritmo concentra sus esfuerzos en aquellas regiones que, según la evidencia actual, parecen ofrecer las mejores perspectivas de éxito. Su lógica es pragmática: utilizar el conocimiento existente para progresar rápidamente hacia una solución.

A primera vista, podría parecer que una de estas estrategias es claramente superior a la otra. Sin embargo, ambas presentan ventajas y limitaciones. Un sistema excesivamente explorador puede terminar dispersando sus recursos en una gran cantidad de alternativas poco relevantes, prolongando innecesariamente el proceso de búsqueda. Por el contrario, un sistema excesivamente explotador corre el riesgo de quedar atrapado en soluciones mediocres debido a su incapacidad para considerar opciones diferentes.

En consecuencia, el verdadero desafío no consiste en elegir entre exploración o explotación, sino en encontrar un equilibrio adecuado entre ambas. La calidad final de una búsqueda depende en gran medida de cómo se gestione esta tensión.

Es común asociar este dilema principalmente al aprendizaje por refuerzo. En ese contexto, un agente debe decidir continuamente si ejecuta acciones conocidas que producen buenas recompensas o si experimenta con nuevas acciones para descubrir oportunidades potencialmente mejores. Sin embargo, el mismo principio aparece también en los problemas clásicos de búsqueda y optimización, aunque adopta una forma diferente (Murphy, 2022).

En los métodos de búsqueda heurística no existe un agente aprendiendo mediante prueba y error a lo largo del tiempo. El espacio de estados ya se encuentra definido y las reglas del problema son conocidas. Por esta razón, el equilibrio entre exploración y explotación no surge de decisiones experimentales sucesivas, sino que queda incorporado directamente en la función de evaluación utilizada para guiar la búsqueda.

Balance analítico en la función de evaluación

En los métodos de búsqueda informada, las dos fuerzas del dilema se reflejan directamente en la función de evaluación utilizada para seleccionar los estados que serán expandidos:

$$f(n) = g(n) + h(n)$$

Esta ecuación combina dos fuentes de información complementarias. Por un lado, g(n) representa el costo acumulado desde el estado inicial hasta el nodo actual. Por otro, h(n) estima el costo restante necesario para alcanzar la meta desde ese mismo estado (Russell y Norvig, 2016).

Desde una perspectiva conceptual, la heurística h(n) puede interpretarse como una fuerza orientada hacia la explotación. Su función consiste en dirigir la búsqueda hacia aquellos estados que parecen más cercanos al objetivo, aprovechando el conocimiento disponible sobre el estructura del problema. Cuanto menor sea la estimación heurística, más atractivo resultará el estado para el algoritmo.

Cuando una estrategia de búsqueda se apoya exclusivamente en esta información, como ocurre en la búsqueda voraz, el sistema adopta una postura marcadamente codiciosa. Las decisiones se toman únicamente en función de la proximidad aparente a la meta. El algoritmo explota agresivamente la información heurística disponible, avanzando siempre hacia el estado que parece ofrecer el progreso más rápido. Esta característica suele producir búsquedas muy eficientes, pero también las vuelve vulnerables a errores de estimación y a estructuras engañosas dentro del espacio de estados (Poole y Mackworth, 2017).

El costo acumulado g(n), en cambio, introduce una perspectiva diferente. Este componente no intenta predecir el futuro ni estimar distancias restantes. Su función es registrar el costo real que ya fue necesario invertir para llegar hasta el estado actual. Cada movimiento realizado queda reflejado en este valor.

Al incorporarse dentro de la función de evaluación, el costo acumulado actúa como un mecanismo de regulación. A medida que una trayectoria se vuelve más larga o costosa, el crecimiento progresivo de g(n) incrementa el valor total de evaluación y reduce la prioridad de continuar profundizando en esa dirección. Como consecuencia, caminos que inicialmente parecían menos atractivos pueden volver a competir por la atención del algoritmo.

Este efecto produce un equilibrio natural entre exploración y explotación. La heurística impulsa la búsqueda hacia regiones prometedoras del espacio de estados, mientras que el costo acumulado evita que el algoritmo se comprometa de manera prematura con una única alternativa. Ninguno de los dos componentes domina completamente el proceso de decisión. Por el contrario, ambos participan simultáneamente en la evaluación de cada estado.

Desde esta perspectiva, la función f(n) puede interpretarse como un mecanismo de arbitraje entre dos preguntas fundamentales. La primera es: ¿qué tan cerca parece estar la meta? La segunda es: ¿cuánto ha costado llegar hasta aquí? La heurística responde a la primera pregunta; el costo acumulado responde a la segunda. La evaluación final integra ambas perspectivas en una única medida de calidad.

El resultado es una estrategia de búsqueda más equilibrada. En lugar de perseguir ciegamente la mejor estimación local o de expandir indiscriminadamente todas las alternativas posibles, el algoritmo distribuye sus esfuerzos considerando simultáneamente el progreso realizado y la promesa futura de cada estado. De esta manera, la tensión entre exploración y explotación deja de ser una decisión intuitiva y pasa a formar parte de la propia estructura matemática que guía la búsqueda.

> Este principio constituye una de las ideas más importantes detrás de las búsquedas heurísticas modernas. La calidad de la solución obtenida no depende únicamente de la precisión de la heurística ni exclusivamente del costo acumulado, sino de la manera en que ambos componentes se combinan para equilibrar eficiencia, profundidad de análisis y capacidad de descubrimiento dentro del espacio de estados (Russell y Norvig, 2016).

A modo de reflexión

Piensa en un proceso de toma de decisiones secuenciales de tu carrera: un diagnóstico iterativo, una investigación de seguridad, el diseño de un experimento en el lab, la optimización de un robot o el desarrollo de un producto.

¿En qué momentos del proceso estás explorando activamente (buscando nueva información, probando hipótesis nuevas, tolerando incertidumbre) y en cuáles estás explotando (usando lo que ya sabes para obtener el mejor resultado posible)?

Aplicación al caso

Vuelve al sistema de protección de infraestructura eléctrica con el que se abrió la lectura, ahora con todas las herramientas conceptuales encima de la mesa.

El primer punto era la inviabilidad de la búsqueda exhaustiva. Con diez mil dispositivos, el número de órdenes posibles de inspección es 10.000!, un número que no tiene ninguna importancia práctica más allá de ilustrar que ese camino está completamente cerrado. No es que sea lento, es

que nunca terminaría. Cualquier método que garantice explorar todo el espacio queda descartado por una razón matemática de fondo.

Lo segundo que el caso ilustró es que la optimalidad no siempre es el objetivo correcto. El sistema no está diseñado para encontrar exactamente y solo los dispositivos comprometidos. Está diseñado para no perderse ninguno. Esa diferencia implica una decisión explícita sobre la función heurística, ya que se prefiere una heurística que sobreestime el riesgo (lo que técnicamente va en contra de admisibilidad) para garantizar que ningún nodo peligroso quede en el fondo de la lista. La heurística inadmisible es una elección de diseño, no un error.

Aquí es donde entra la reflexión que se dejó abierta: ¿qué pasaría en una red hospitalaria? La respuesta es que el diseño de la función heurística es fundamentalmente una decisión sobre el costo relativo de los errores posibles. En la red eléctrica, un falso positivo cuesta atención humana innecesaria; un falso negativo puede costar un apagón. En una red hospitalaria, un falso positivo puede interrumpir cirugías; un falso negativo puede ser una brecha de datos críticos. Cambiar el dominio cambia la función de costo, y cambiar la función de costo cambia cómo debería diseñarse la heurística.

El tercer elemento es el dilema exploración-explotación. El sistema del caso opera bajo presión de tiempo extrema, lo que lo orienta fuertemente hacia la explotación: usar la información disponible para priorizar rápido, sin detenerse a explorar alternativas. Pero si el ataque tiene un vector desconocido, esa postura explotadora puede hacer que el sistema nunca lo detecte porque no está buscando en esa dirección. Un sistema más sofisticado podría reservar una fracción de sus recursos para inspeccionar dispositivos de bajo puntaje pero con perfiles de comportamiento atípicos; eso sería introducir exploración controlada dentro de un sistema fundamentalmente explotador.

Finalmente, el caso conecta con el principio de la función $f(n) = g(n) + h(n)$. El puntaje que asigna el sistema combina el costo real de las alertas ya disparadas, $g(n)$, con la estimación heurística del daño potencial considerando la importancia del dispositivo en la red, $h(n)$.

Sin el componente $g(n)$, el sistema priorizaría siempre los nodos más críticos de la red, aunque no hubiera ningún indicio de compromiso. Sin $h(n)$, quedaría atrapado en dispositivos con muchas alertas menores, ignorando una subestación central que todavía no disparó ninguna alarma, pero que está a un movimiento del atacante. La función $f(n)$ integra ambas perspectivas, y esa integración es exactamente lo que le da valor operativo al sistema.

Actividades de repaso de lecturas

ACTIVIDADES DE REPASO Y REFUERZO

PREGUNTAS VERDADERO/FALSO

Toda heurística admisible es también consistente, por lo que ambas propiedades pueden considerarse equivalentes a la hora de evaluar la calidad de una función heurística.

Es verdadero, porque si una heurística nunca sobreestima el costo real, entonces satisface la condición de consistencia en todos los nodos del grafo.

Es falso, porque la relación de implicación va en sentido contrario, ya que toda heurística consistente es admisible, pero una heurística puede ser admisible sin ser consistente. Son propiedades distintas con exigencias distintas.

Referencias

Hart, P. E., Nilsson, N. J., Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

Masip Rodó, D., Llorà, X., Domingo, J. (2015). Algorísmica i complexitat (2a ed.). Editorial UOC.

Murphy, K. P. (2022). Probabilistic machine learning: An introduction. MIT Press.

Poole, D. L., Mackworth, A. K. (2017). Artificial intelligence: Foundations of computational agents (2nd ed.). Cambridge University Press.

Russell, S., Norvig, P. (2016). Artificial intelligence: A modern approach (3rd ed.). Pearson.